Análisis: Migrar App Móvil TTPN a PWA

Fecha: 2026-04-10
Contexto: App Android nativa actual vs Progressive Web App (Quasar PWA)

¿Qué es una PWA en el contexto de TTPN?

Una PWA es una aplicación web que se "instala" en el dispositivo y se comporta como una app nativa: ícono en pantalla de inicio, funciona sin conexión parcialmente, accede a cámara y GPS. En el caso de TTPN, Quasar (el framework del frontend actual) **soporta PWA nativamente** — significa que el código del panel web podría convertirse en la app móvil sin reescribir todo desde cero.

Resumen ejecutivo

Dimensión	App Android Nativa	PWA (Quasar)
Actualizaciones	Requiere nueva APK + descarga	Automáticas al abrir la app
Desarrollo	Java/Kotlin separado del FE web	Un solo codebase con el panel admin
Notificaciones push	FCM nativo — completo	Web Push — funciona en Android, **limitado en iOS**
Acceso a hardware	Completo (GPS, cámara, sensores)	Parcial (GPS y cámara sí, sensores avanzados no)
Funcionamiento offline	Total (SQLite local posible)	Parcial (Service Worker + caché)
Distribución	Google Play Store	URL directa / PWA install prompt
Rendimiento	Alto (compilado nativo)	Medio-alto (depende del dispositivo y browser)
Costo de mantenimiento	Alto (equipo móvil separado)	Bajo (mismo equipo que FE web)

LO QUE SE PIERDE al migrar a PWA

1. Notificaciones push completas en iOS 🟡 *(impacto reducido)*

Impacto en TTPN: Reducido — el sistema de alertas con campana en BE/FE cubre el caso de uso principal.
Detalle: iOS Safari soporta Web Push desde iOS 16.4 (2023), pero requiere que el usuario haya "instalado" la PWA desde Safari. La adopción es inconsistente.
Mitigación real: El módulo de Alertas ya implementado en BE y FE funciona como bandeja de

notificaciones persistente — el chofer abre la app y la campana le indica qué tiene pendiente, sin depender de push. Es el mismo patrón que usan WhatsApp Web, Slack web, etc.

2. Firebase Cloud Messaging (FCM) nativo 🟡 *(impacto reducido)*

Impacto en TTPN: Reducido por el sistema de alertas con campana.

Detalle: FCM requiere app nativa. Las PWA usan Web Push (VAPID), que funciona bien en Android/Chrome pero no en iOS sin instalación previa.

Estrategia para TTPN:

- **Campana de alertas (ya en desarrollo)** → notificaciones dentro de la app al abrirla — cubre el 90% de los casos
- **Web Push** → notificación del sistema operativo cuando la app está cerrada — cubre Android, limitado en iOS
- **SMS/email** → fallback para alertas críticas si ninguna de las anteriores llega

Esta combinación elimina la dependencia de FCM sin perder cobertura operativa.

3. Acceso a sensores avanzados del dispositivo 🟡

Impacto en TTPN: Acelerómetro, giroscopio, Bluetooth — si alguna feature futura los necesita.
Detalle: La app actual no parece usarlos, por lo que el impacto es bajo hoy pero limita opciones futuras.

4. Funcionamiento offline profundo 🟡

Impacto en TTPN: Un chofer en zona sin cobertura que necesita registrar un travel_count.
Detalle: La app Android puede usar SQLite local para guardar datos offline y sincronizar después. Una PWA puede usar IndexedDB + Service Worker para lo mismo, pero es más complejo de implementar y tiene límites de almacenamiento más estrictos impuestos por el browser.
Mitigación: Si los choferes siempre tienen conexión (zonas urbanas), no es un problema real.

5. Rendimiento en dispositivos Android antiguos 🟡

Impacto en TTPN: Si la flota usa teléfonos de gama baja (Android 8-10, 2-3 GB RAM).
Detalle: Una PWA en Chrome en un teléfono de 2019 puede ser notablemente más lenta que una app nativa compilada. Especialmente en listas largas, formularios pesados, o cuando hay muchos travel_counts.

6. Presencia en Google Play Store 🟡

Impacto en TTPN: Distribución interna vs pública.
Detalle: Si TTPN usa Google Play para distribuir la app a choferes, perder esa presencia significa gestionar actualizaciones manualmente (URL + instrucciones de instalación). Sin embargo, las PWA en Android se pueden empaquetar y publicar en Play Store usando **TWA (Trusted Web Activity)** — Quasar soporta esto via Capacitor.

7. Acceso nativo a Bluetooth y NFC 🔵

Impacto en TTPN: Bajo actualmente — no hay evidencia de uso.

LO QUE SE GANA al migrar a PWA

1. Un solo equipo para web y móvil ✅

Impacto: El equipo que mantiene el panel admin Quasar puede también mantener la app móvil. No se necesita un desarrollador Java/Kotlin separado.
En TTPN: Ahora mismo hay al menos un desarrollador móvil independiente del equipo Rails/Vue. Con PWA, ese trabajo se fusiona.

2. Actualizaciones instantáneas sin APK 🚀

Impacto en TTPN: Hoy, corregir un bug en la app requiere: compilar APK → subir a Play Store → esperar aprobación (1-3 días) → esperar que los choferes actualicen.
Con PWA: Hacer deploy del FE en Heroku/Vercel → la próxima vez que el chofer abre la app, ya tiene la versión nueva. Sin Play Store, sin esperar.

3. Reutilizar componentes del panel admin 🔧

Impacto: Los formularios de travel_count, gas_charge, vehicle_check ya existen o existirán en el panel web. En una PWA se pueden reutilizar directamente, adaptados a pantalla pequeña con responsive design Quasar.

4. Mantenimiento del módulo "Acceso API" se vuelve innecesario 🔧

Impacto: Si la app móvil es una PWA que usa directamente la sesión web (mismo JWT, mismos endpoints), el módulo de Acceso API (que gestiona tokens para la app externa) se simplifica enormemente o desaparece.

5. Control de versiones centralizado 📋

Impacto: El módulo de Versiones en Settings (que acabamos de arreglar) cobra mucho más sentido con una PWA — se puede forzar a los usuarios a actualizar redireccionando desde el Service Worker.

6. GPS y cámara disponibles vía browser ✅

Impacto: Los features actuales de la app (coordenadas GPS en gas_charge, fotos en vehicle_check) funcionan perfectamente en PWA:

```
// GPS: navigator.geolocation.getCurrentPosition(...)
// Cámara: navigator.mediaDevices.getUserMedia({ video: true })
// O input file con capture="environment"
```

7. Sin dependencia de Android SDK / versiones de Java ✅

Impacto: El proyecto Android tiene dependencias en versiones específicas de Android SDK, Gradle, Java. Con PWA, solo hay que mantener el stack web (Node, Quasar, Vue).

Análisis por módulo de la app actual

Módulo	¿Funciona en PWA?	Notas
Login / Sesión	✅ Completo	JWT funciona igual
Listado de bookings	✅ Completo	Solo HTTP GET
Creación de travel_count	✅ Completo	Form web adaptado a móvil
Auto-sugerencia de planta por GPS	✅ Completo	Haversine en JS es idéntico — ver §Geolocalización
Mapa de gasolineras	🟡 Cambia	Google Maps SDK → Google Maps JS API o Leaflet
Carga de gasolina	✅ Completo	GPS con `navigator.geolocation`
Fotos vehicle_check	✅ Completo	`<input capture="environment">`
Notificaciones de nuevo booking	🟡 Parcial	Web Push funciona en Android; iOS limitado
Funcionamiento sin conexión	🟡 Parcial	IndexedDB + Service Worker — requiere desarrollo
Firebase token (FCM)	🔴 Cambia	Se reemplaza por Web Push VAPID

Geolocalización y auto-sugerencia de planta

Esta funcionalidad merece análisis detallado porque es crítica para la operación del chofer.

Cómo funciona en la app Android (`ConteoViajes.java`)

1. Al abrir el formulario de travel_count, la app obtiene la posición del dispositivo via `LocationManager`
2. Descarga todas las plantas de clientes con sus coordenadas (`GASTO_SELECT_CLIENT_BRANCH_OFFICES_LAT_LNG.php`)
3. Calcula la distancia a cada planta usando la **fórmula de Haversine** (distancia en km sobre la esfera terrestre)
4. Si alguna planta está a **≤ 0.3 km (300 metros)**, la selecciona automáticamente en el spinner
5. Si varias plantas están dentro del radio, selecciona la más cercana

```
// Algoritmo en ConteoViajes.java:
if (dDistanciaEntrePosiciones <= 0.3) {         // Radio: 300 metros
    if (iContador != 0 && dDistanciaEntrePosiciones < dComparar) {
        sClienteSeleccioando = listCLIENTES.get(j + 1);  // Auto-
selecciona la más cercana
    }
}
// Si la planta no tiene lat/lng → distancia = 1.0 → nunca se auto-
selecciona
```

Equivalente en PWA/JavaScript — Funciona igual ✅

```
// Haversine idéntico en JS:
function calcularDistancia(lat1, lon1, lat2, lon2) {
  const R = 6371
  const dLat = (lat2 - lat1) * Math.PI / 180
  const dLon = (lon2 - lon1) * Math.PI / 180
  const a = Math.sin(dLat/2) ** 2 +
            Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI /
180) *
            Math.sin(dLon/2) ** 2
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a))
}

// GPS del browser:
navigator.geolocation.getCurrentPosition(pos => {
  const { latitude, longitude } = pos.coords
  const branches = await api.get('/api/v1/client_branch_offices?
fields=id,nombre,lat,lng')

  const cercanas = branches
    .filter(b => b.lat && b.lng)
    .map(b => ({ ...b, distancia: calcularDistancia(latitude, longitude,
b.lat, b.lng) }))
    .filter(b => b.distancia <= 0.3)
    .sort((a, b) => a.distancia - b.distancia)

  if (cercanas.length > 0) {
    formData.client_branch_office_id = cercanas[0].id  // Auto-selecciona
    mostrarSugerencia(cercanas[0].nombre)
  }
})
```

Endpoint Rails — ya disponible ✅

La app llama a `GASTO_SELECT_CLIENT_BRANCH_OFFICES_LAT_LNG.php`. El equivalente Rails ya existe y devuelve `lat` y `lng`:

```
GET /api/v1/client_branch_offices
GET /api/v1/client_branch_offices?client_id=42
```

El controlador `ClientBranchOfficesController#index` ya serializa `lat` y `lng` en cada registro. El FE también lo usa actualmente en dropdowns de varios modales. No requiere trabajo adicional en BE.

Regla de negocio a documentar

El radio de 300 metros está hardcodeado en el APK. Considerar:

- Moverlo a una constante en el backend (configurable por admin)

- Agregar campo `radio_metros` a `client_branch_offices` para plantas con geofence diferente

Mapa de gasolineras

La app `geolocalizacion.java` usa **Google Maps SDK nativo** para mostrar gasolineras en un mapa interactivo. En PWA hay dos opciones:

- **Google Maps JavaScript API** — misma funcionalidad, licencia igual, se integra en Vue/Quasar con `vue3-google-map`
- **Leaflet + OpenStreetMap** — gratuito, sin restricciones de cuota, calidad similar para uso interno

Ambas soportan marcadores, polilíneas y detección de posición del usuario.

Opción intermedia: Quasar + Capacitor

Quasar ofrece **Capacitor** como puente: escribe la app en Vue/Quasar y la empaqueta como APK nativo para Android e iOS. Se obtiene lo mejor de ambos mundos:

Característica	PWA pura	Quasar + Capacitor
Un solo codebase	✅	✅
FCM nativo	❌	✅
Distribución Play Store	Via TWA	✅ Nativo
Cámara nativa	✅ (browser)	✅ (nativo)
Offline profundo	🟡	✅
Actualizaciones automáticas	✅	❌ (requiere nueva APK)
Complejidad	Baja	Media

Recomendación: Si las notificaciones push son críticas para la operación (los choferes DEBEN recibir avisos en tiempo real), usar **Quasar + Capacitor**. Si las notificaciones son opcionales o la flota es 100% Android con Chrome actualizado, **PWA pura** es suficiente.

Estimación de esfuerzo de migración

Opción A — PWA pura (sin Capacitor)

Tarea	Estimado
Configurar Quasar PWA mode + Service Worker	1–2 días
Adaptar layouts existentes a pantalla móvil (responsive)	3–5 días
Módulo de Login móvil (reusar composable de auth)	1 día
Módulo Travel Counts (formulario + listado)	3–5 días
Módulo Gas Charge con GPS	2–3 días

Tarea	Estimado
Módulo Vehicle Checks con cámara	2–3 días
Módulo Bookings del chofer	2–3 días
Web Push (reemplazo de FCM)	2–3 días
Testing y ajustes en dispositivos reales	3–5 días
Total	**19–30 días hábiles**

Opción B — Quasar + Capacitor (APK nativo)

Todo lo anterior más:

Tarea extra	Estimado
Configurar Capacitor + plugins nativos	1–2 días
Integración FCM nativa (Capacitor Push Notifications)	1–2 días
Build pipeline para APK	1 día
Total adicional	**3–5 días más**

Recomendación final

Para TTPN, la opción recomendada es Quasar + Capacitor, por estas razones:

1. **Las notificaciones son críticas** — un chofer que no recibe aviso de un booking nuevo tiene impacto operativo directo.
2. **El codebase ya existe** — el panel admin Quasar comparte componentes, composables y servicios. El esfuerzo real es adaptar vistas, no reescribir lógica.
3. **Elimina el equipo móvil separado** — el mismo desarrollador que mantiene el panel web puede hacer el build de Capacitor.
4. **Play Store opcional** — se puede distribuir como APK directo sin pasar por Play Store para despliegues internos.

Lo que se pierde es mínimo: el código Java existente queda obsoleto, pero ya era un costo de mantenimiento. Todo lo valioso (tablas, triggers, lógica de negocio) vive en Ruby/PostgreSQL y no cambia.

Prerrequisitos antes de migrar a PWA

1. **Fase 2 de migración móvil completada** — la app debe estar usando Rails REST API (no PHP)
2. **Todos los endpoints Rails implementados** — ver `APP_MOVIL_BE_AJUSTES.md`
3. **Quasar FE estabilizado** — el panel admin debe estar maduro antes de añadir la capa móvil
4. **Decisión sobre notificaciones** — definir si Web Push es suficiente o se necesita Capacitor